

Université de Cergy-Pontoise

Rapport

Pour le projet UE OS

Licence d'Informatique troisième année

Sur le sujet

Gestionnaire des Fichiers



Rédigé Par :

BOUKILI Inas, MEZIANE Fella, OFFIONG Dara

Groupe C3

01 avril 2025

1. INTRODUCTION -----	3
1.1 Contexte-----	3
1.2 Objectifs-----	3
2. CONCEPTION ET IMPLEMENTATION -----	3
2.1 Fonctionnalités implémentées -----	3
3. Algorithmes -----	5
3.1 Structures de données -----	5
3.2 Algorithmes principaux : -----	7
4. TESTS ET RESULTATS -----	8
5. CONCLUSION-----	14

1. INTRODUCTION

1.1 Contexte

Les systèmes de fichiers représentent une composante fondamentale des systèmes d'exploitation. Sous UNIX, le paradigme « tout est fichier » implique que chaque élément du système — qu'il s'agisse de fichiers classiques, de répertoires, de périphériques ou même de processus — est manipulé de manière unifiée via des descripteurs de fichiers. Ce modèle impose une gestion rigoureuse des métadonnées, des droits d'accès et de l'organisation hiérarchique.

1.2 Objectifs

L'objectif principal est de développer une application capable d'exécuter les opérations suivantes sur une partition simulée :

1. **Création et suppression de fichiers et de répertoires**
Permettre à l'utilisateur de structurer les données via des opérations analogues à **touch**, **mkdir** et **rm**.
2. **Copie et déplacement**
Reproduire les commandes **cp** et **mv** afin de dupliquer ou déplacer des fichiers/répertoires à différents emplacements.
3. **Gestion des permissions d'accès.**
4. **Support des liens physiques et symboliques** : Implémenter les commandes **ln** et **ln -s**, afin de créer des liens physiques ou symboliques vers d'autres fichiers.
5. **Optimisation de l'espace disque** : Mettre en place une gestion efficace de l'espace libre pour éviter la fragmentation.

2. CONCEPTION ET IMPLEMENTATION

2.1 Fonctionnalités implémentées

1. **Répertoires** : Les répertoires sont représentés par des inodes spéciaux qui contiennent une liste d'enfants - autres fichiers ou sous-répertoires.

Code associé :

```
typedef struct
{
    char filename[MAX_FILENAME];
    int is_directory;
    int is_symlink;
    int first_block;
    int size;
    int parent_dir;
    mode_t permissions;
    time_t creation_time;
    time_t modification_time;
    int link_count;
    int entries[MAX_DIR_ENTRIES]; // Tableau des inodes des entrées
    int entry_count;
    int num_entries;
} Inode;
```

2. Liens symboliques et liens physiques : Le système de fichiers gère les liens symboliques, qui sont des références vers d'autres fichiers ou répertoires. Les liens physiques, les hard links sont également supportés, permettant de référencer un fichier à partir de plusieurs emplacements sans duplication de contenu.

Code associé :

Dans gestionnaire.h :

```
int fs_symlink(const char *target, const char *linkpath);

char *resolve_symlink(int inode_idx);
```

Dans gestionnaire.c :

```
int fs_symlink(const char *target, const char *linkpath)
{
    if (target == NULL || linkpath == NULL || strlen(target) == 0 || strlen(linkpath) == 0)
    {
        fprintf(stderr, "Erreur: Paramètres invalides\n");
        errno = EINVAL;
        return -1;
    }
}
```

3. Historique des backups : Le système permet la gestion de sauvegardes, où chaque modification du système de fichiers est enregistrée, permettant ainsi la restauration à une version antérieure.

Code associé :

Dans gestionnaire.h :

```
// Backup system
void create_backup();
void list_backups();
int restore_backup(int index);
void fs_truncate(int fd, off_t new_size);
void fs_ls_l(const char *path);
int add_entry_to_dir(int dir_inode, int new_inode);
void update_all_hardlinks(int inode_num, size_t new_size, time_t mtime);
#endif
```

Dans gestionnaire.c :

```
void create_backup()
{
    // Vérifier si on a atteint le nombre maximum de sauvegardes
    if (backup_count >= MAX_BACKUPS)
    {
        // Décaler les backups pour faire de la place (FIFO)
        for (int i = 0; i < MAX_BACKUPS - 1; i++)
        {
            memcpy(&backups[i], &backups[i + 1], sizeof(FileSystem));
            backup_times[i] = backup_times[i + 1];
        }
        backup_count--;
    }

    // Créer le nouveau backup
    memcpy(&backups[backup_count], &fs, sizeof(FileSystem));
    backup_times[backup_count] = time(NULL);
    backup_count++;

    if (DEBUG)
    {
        char time_str[50];
        strftime(time_str, sizeof(time_str), "%Y-%m-%d %H:%M:%S", localtime(&backup_times[backup_count - 1]));
        printf("[BACKUP] Créé à %s (total: %d)\n", time_str, backup_count);
    }
}
```

3. Algorithmes

3.1 Structures de données

1. Inodes

Un inode est une structure qui représente un fichier, un répertoire ou un lien symbolique dans le système de fichiers. Chaque inode contient les métadonnées nécessaires à la gestion de l'entité :

- Le nom du fichier ou du répertoire - **"filename"**.
- Le type : fichier, répertoire - **is_directory** ou lien symbolique - **"is_symlink"**.
- L'index du répertoire parent - **"parent_dir"**.
- Le premier bloc de données alloué - **"first_block"**.
- La taille du fichier - **"size"**.
- Les permissions d'accès - **"mode_t permissions"**.
- Les dates de création et de modification - **"creation_time, modification_time"**.
- Le nombre de liens physiques - **"link_count"**.

2. Table des i-nodes

L'ensemble des inodes est stocké dans un tableau fixe **inodes [MAX_FILES]**, géré au sein de la structure globale **FileSystem**. Chaque inode est identifié par un indice unique. Un tableau parallèle **free_inodes[MAX_FILES]** indique l'état de chaque inode (0 = libre, 1 = utilisé), facilitant l'allocation et la libération des inodes.

3. Bitmap de blocs libres

Un bitmap est utilisé pour gérer l'espace libre sur la partition simulée. Chaque bit représente un bloc de données :

- 0 : bloc libre
- 1 : bloc occupé

Ce mécanisme permet une allocation rapide et optimisée des blocs.

4. Liste chaînée des fichiers ouverts

Chaque bloc est représenté par la structure suivante :

```
// Type definitions
typedef struct Block {
    char data[BLOCK_SIZE];
    int next;
} Block;
```

Une liste chaînée conserve les informations sur les fichiers actuellement ouverts par le système. Elle facilite la gestion des accès concurrents et garantit une libération correcte des ressources lors de la fermeture.

5. Descripteurs de fichiers ouverts

```
typedef struct FileDescriptor {  
    int inode_idx;  
    off_t position;  
} FileDescriptor;
```

Elle permet de conserver la position actuelle dans le fichier et de référencer l'inode correspondant "inode_idx". Une liste chaînée ou un tableau de ces descripteurs permettrait la gestion simultanée de plusieurs fichiers ouverts.

Gestion des liens symboliques et physiques

Les liens symboliques sont identifiés par le champ **is_symlink** dans l'inode. La cible du lien est enregistrée dans le bloc de données associé. Lorsqu'un lien symbolique est ouvert, le système suit récursivement la cible jusqu'à atteindre l'inode réel (avec une limite de profondeur **MAX_SYMLINK_DEPTH** pour éviter les boucles infinies).

Les liens physiques sont gérés via le champ **link_count**, permettant à plusieurs chemins de référencer un même inode et donc partager les mêmes données sur disque.

3.2 Algorithmes principaux :

- **Allocation de blocs** : L'algorithme d'allocation de blocs dans ce projet utilise une stratégie de premier bloc libre trouvé, où le premier bloc disponible dans le bitmap des blocs libres est alloué lorsqu'un fichier est créé. Cette approche simple garantit une allocation rapide des blocs. Dans notre implémentation actuelle, les blocs sont alloués de manière à être aussi contigus que possible, ce qui améliore l'efficacité des accès aux fichiers et réduit le temps de lecture et d'écriture.
- **Suppression de fichiers** : Lorsqu'un fichier est supprimé, les blocs de données alloués à ce fichier sont libérés et marqués comme libres dans le bitmap. De plus, les i-nodes associés au fichier sont réinitialisés. Cela permet de récupérer l'espace mémoire utilisé par le fichier et de le réutiliser pour d'autres fichiers.
- **Gestion des permissions** : La gestion des permissions est simulée à travers les appels systèmes `chmod` et `chown`. Ces algorithmes permettent de contrôler les droits d'accès aux fichiers et répertoires. Par exemple, `chmod` modifie les permissions de lecture, écriture et exécution, tandis que `chown` change le propriétaire et le groupe d'un fichier.
- **Optimisation de l'espace libre** : Un mécanisme de compactage a été implémenté pour minimiser la fragmentation de l'espace disque. Ce processus est effectué périodiquement, en réorganisant les blocs de données pour optimiser l'utilisation de l'espace libre. L'algorithme de défragmentation déplace les blocs de données afin de les stocker de manière contiguë, ce qui réduit les espaces vides entre les fichiers. Cette opération permet non seulement d'améliorer

l'utilisation de l'espace disque, mais aussi d'accélérer les temps d'accès aux fichiers en facilitant la lecture séquentielle des données.

4. TESTS ET RESULTATS

1. Les tests préliminaires :

a. Affichage des commandes avec "help" :

```
=== Système de fichiers UNIX simplifié - Interface Shell ===
Tapez 'help' pour voir les commandes disponibles

Erreur: Fichier corrompu
Système initialisé. Blocs libres: 1024, Inodes libres: 63
/$ help
===== SYSTÈME DE FICHIERS UNIX SIMPLIFIÉ =====
Commandes disponibles:
ls [chemin]           - Liste le contenu du répertoire
mkdir <nom>           - Crée un répertoire
rmdir <nom>           - Supprime un répertoire (doit être vide)
cd <chemin>           - Change de répertoire
pwd                  - Affiche le répertoire courant
touch <nom>          - Crée un fichier vide
cat <nom>            - Affiche le contenu d'un fichier
echo <text> > <nom>   - Écrit du texte dans un fichier
rm <nom>             - Supprime un fichier
chmod <mode> <nom>   - Change les permissions (ex: 755)
ln -s <cible> <lien>  - Crée un lien symbolique
ln <cible> <lien>     - Crée un lien physique
tree [--inodes] [chemin] - Affiche l'arborescence des fichiers
format              - Formate le système de fichiers
save                - Sauvegarde l'état actuel
load                - Charge la dernière sauvegarde
stats               - Affiche les statistiques
debug              - Affiche les infos de débogage
defrag              - Défragmente le système de fichiers
exit                - Quitte le programme
help                - Affiche cette aide
```

b. Affichage de contenu dans le répertoire courant et toutes les informations pertinentes telle que la taille, les permissions etc.

```
=====
/$ ls
Contenu du répertoire courant '/':
Type Permissions  Taille Nom
-----
d      rwxr-xr-x      0 /

Total: 1 élément(s)
```


c. Création de répertoire et fichier, navigation d'un vers un autre et écriture dans le fichier vide créer.

```
/$ ls -l
Contenu du répertoire courant '/':
Type Liens Permissions Taille Date Nom
----
d      1  rwxr-xr-x      0 Apr 02 23:56 /

Total: 1 élément(s)
/$ mkdir test
/$ cd test
/test$ pwd
/test
/test$ touch fichier1
Fichier 'fichier1' créé avec succès
/test$ echo "Bonjour" > fichier1
Début écriture dans fichier1 (position: 0, taille: 9 octets)
Allocation du premier bloc...
Écriture réussie: 9 octets écrits dans fichier1
Début écriture dans fichier1 (position: 9, taille: 1 octets)
Écriture réussie: 1 octets écrits dans fichier1
Texte écrit avec succès dans 'fichier1' (10 octets)
```

2. Manipulation des fichiers :

a. Affichage du contenu dans le fichier1, changement des permissions et éditer le fichier1

```
/test$ cat fichier1
Contenu du fichier 'fichier1':
-----
"Bonjour"
-----
Fin du fichier (total: 10 octets)
/test$ chmod 755 fichier1
/test$ edit fichier1
Début écriture dans fichier1 (position: 0, taille: 43 octets)
Écriture réussie: 43 octets écrits dans fichier1
Sauvegardé 43 octets dans fichier1
```

b. copier le fichier1 vers un fichier2

```
/test$ cp fichier1 fichier2
Début écriture dans fichier2 (position: 0, taille: 43 octets)
Allocation du premier bloc...
Écriture réussie: 43 octets écrits dans fichier2
```

c. Déplacement d'un fichier1 vers un fichier3 quelconque

```
Fin du fichier (total: 43 octets)
/test$ mv fichier1 fichier3
/test$ ls
Contenu du répertoire courant '/test':
Type Permissions  Taille Nom
-----
-   rwxr-xr-x      43 fichier3
-   rw-r--r--      43 fichier2

Total: 2 élément(s)
```

d. Création d'un lien symbolique

```
/test$ ln -s fichier2 lien_sym
Lien symbolique créé: lien_sym -> fichier2
/test$ cat lien_sym
[DEBUG] Résolution lien 4 -> 'fichier2'
Contenu du fichier 'lien_sym':
-----
"Bonjour"
i am currently editing this file
-----
Fin du fichier (total: 43 octets)
```

3. Affichage de l'arborescence :

a. Affichage du Tree :

```
/test$ tree
.
test
├── fichier2
├── fichier3
├── lien_phys
└── lien_sym[DEBUG] Résolution lien 4 -> 'fichier2'
-> fichier2
/test$ tree --inodes
[1] .
[1] test
├── [3] fichier2
├── [2] fichier3
├── [5] lien_phys
└── [4] lien_sym[DEBUG] Résolution lien 4 -> 'fichier2'
-> fichier2
```

4. D'autres éléments clés :

a. Supprimer un répertoire et un fichier

```
/test$ rm fichier3
[BACKUP] Créé à 2025-04-03 00:11:24 (total: 2)
[RELOC] Moving block 2 to 0
CORRECTION: Incohérence inodes (compteur:56 vs réel:57)
[COMPACT] Compacted block 2 to 0
Fichier 'fichier3' supprimé (backup 1 créé)
/test$ rmdir sousdir
/test$ ls
Contenu du répertoire courant '/test':
Type Permissions  Taille Nom
-----
-   rw-r--r--      43 fichier2
l   rwxrwxrwx       8 lien_sym
-   rw-r--r--      43 lien_phys

Total: 3 élément(s)
```

b. Création des backups

```
/test$ backup
[BACKUP] Créé à 2025-04-03 00:12:11 (total: 3)
Backup créé avec succès (total: 3/5)
/test$ backups

=== BACKUPS DISPONIBLES (3/5) ===
[0] 2025-04-03 00:07:11
[1] 2025-04-03 00:11:24
[2] 2025-04-03 00:12:11
=====
```

c. Restauration du backup

```
=====
/test$ cd ..
/$ restore 0
Système restauré à partir du backup du 2025-04-03 00:07:11
CORRECTION: Incohérence inodes (compteur:56 vs réel:57)
Restauration réussie!
/$ ls
Contenu du répertoire courant '/':
Type Permissions  Taille Nom
-----
d   rwxr-xr-x       0 /
d   rwxr-xr-x       0 test

Total: 2 élément(s)
```

d. Affichage des statistiques et débogage :

N° Groupe : 3

```
/test$ cd ..
/$ stats
===== STATISTIQUES DU SYSTÈME DE FICHIERS =====
Fichiers                : 3
Répertoires             : 3
Liens symboliques       : 1
Blocs utilisés          : 3 / 1024 (0.3%)
Inodes utilisés         : 7 / 64 (10.9%)
Espace utilisé          : 129 octets
Taille moyenne fichier : 43 octets
=====
/$ debug

=== DEBUG DU SYSTEME DE FICHIERS ===
Répertoire courant: '/' (inode 0)
Blocs libres: 1021/1024 (0.3% utilisé)
Inodes libres: 57/64 (10.9% utilisé)

Fichiers ouverts (7/64):

État des 10 premiers blocs:
Bloc 0: UTILISÉ (lié à: -1) [test]
Bloc 1: UTILISÉ (lié à: -1) [fichier2]
Bloc 2: UTILISÉ (lié à: -1) [lien_sym]
Bloc 3: LIBRE
Bloc 4: LIBRE
Bloc 5: LIBRE
Bloc 6: LIBRE
Bloc 7: LIBRE
Bloc 8: LIBRE
```

e. Analyse de la fragmentation :

```
État des 10 premiers blocs:
Bloc 0: UTILISÉ (lié à: -1) [test]
Bloc 1: UTILISÉ (lié à: -1) [fichier2]
Bloc 2: UTILISÉ (lié à: -1) [lien_sym]
Bloc 3: LIBRE
Bloc 4: LIBRE
Bloc 5: LIBRE
Bloc 6: LIBRE
Bloc 7: LIBRE
Bloc 8: LIBRE
Bloc 9: LIBRE

Vérification de cohérence:
Blocs libres: compteur=1021, calculé=1021 | OK
Inodes libres: compteur=57, calculé=57 | OK

=== FRAGMENTATION ANALYSIS ===
Used blocks: 3/1024
Gaps between blocks: 0
Largest gap: 0 blocks
Fragmentation rate: 0.0%
Status: No fragmentation detected
=====
```

f. Réinitialisation :

```
=====
/$ format
ATTENTION: Cette opération effacera toutes les données!
Tapez 'OUI' pour confirmer: OUI

ATTENTION: Cette opération va:
- Réinitialiser TOUTES les données
- Supprimer les 3 backups existants
- Créer un nouveau système de fichiers vierge

Formatage réussi. Système réinitialisé.
1024 blocs et 63 inodes libres.
/$ save
[BACKUP] Créé à 2025-04-03 00:13:21 (total: 1)
Système sauvegardé dans 'filesystem.dat' (1 backups internes disponibles)
/$ exit
[BACKUP] Créé à 2025-04-03 00:13:24 (total: 2)
Système sauvegardé dans 'filesystem.dat' (2 backups internes disponibles)
Au revoir! Le système de fichiers a été sauvegardé
```

5. CONCLUSION

Ce mini-projet nous a permis de mettre en œuvre un système de fichiers simulé en reproduisant les principales fonctionnalités d'un système UNIX : gestion des fichiers et répertoires, liens symboliques et physiques, permissions, sauvegardes, et mécanismes de défragmentation.

Grâce à une organisation efficace du travail et une répartition équitable des tâches, nous avons pu finaliser le projet dans les délais impartis. Chacun a contribué à la réflexion, à l'implémentation et aux tests, assurant un développement cohérent et fonctionnel.

Au-delà de l'aspect technique, ce projet nous a permis de mieux comprendre les enjeux réels liés à la gestion bas niveau des systèmes de fichiers, ainsi que la complexité derrière des commandes UNIX que nous utilisons quotidiennement.